Analysis Report

Share this Report

GC log file: **gc-2017_08_19-15_01.log.0.current**

Duration: **1 hr 10 min 30 sec**

Long Pause

Our analysis tells that your application is suffering from long running GC events. **6 GC events took more than 10 seconds.** Long running GCs are unfavourable for application's performance.

Read our recommendations to **reduce long GC pause (https://blog.gceasy.io/2016/11/22/reduce-long-gc-pauses/)**

Application waiting for resources

It looks like your application is waiting due to lack of compute resources (either CPU or I/O cycles). Serious production applications shouldn't be stranded because of compute resources. In **162 GC event(s)**, 'real' time took more than 'usr' + 'sys' time.

Timestamp	User Time (secs)	Sys Time (secs)	Real Time (secs)
2017-08-19T15:10:59	0.57	0.02	0.8
2017-08-19T15:11:03	0.37	0.13	2.22
2017-08-19T15:11:21	0.29	0.03	1.28
2017-08-19T15:11:34	0.17	0.02	0.68
2017-08-19T15:11:54	0.2	0.03	1.45
2017-08-19T15:12:04	0.13	0.04	0.54
2017-08-19T15:12:14	0.27	0.01	0.73
2017-08-19T15:12:51	0.32	0.01	0.82
2017-08-19T15:12:59	0.2	0.01	0.54
2017-08-19T15:13:42	0.39	0.29	16.35
2017-08-19T15:14:17	0.28	0.05	1.36
2017-08-19T15:14:22	0.31	0.02	0.75
2017-08-19T15:15:36	0.2	0.03	0.97
2017-08-19T15:16:00	0.44	0.07	1.83
2017-08-19T15:16:29	0.45	0.04	0.91
2017-08-19T15:16:52	0.27	0.04	1.48
2017-08-19T15:16:55	0.39	0.01	1.43

2017-08-19T15:16:59	0.79	0.04	1.18
2017-08-19T15:17:19	0.37	0.02	0.72
2017-08-19T15:17:25	0.45	0.02	1.0
2017-08-19T15:17:36	0.28	0.04	0.53
142 more events...			

Read our recommendations to **fix this problem (https://blog.gceasy.io/2016/12/08/real-time-greater-than-user-and-sys-time/)**

System Time greater than User Time

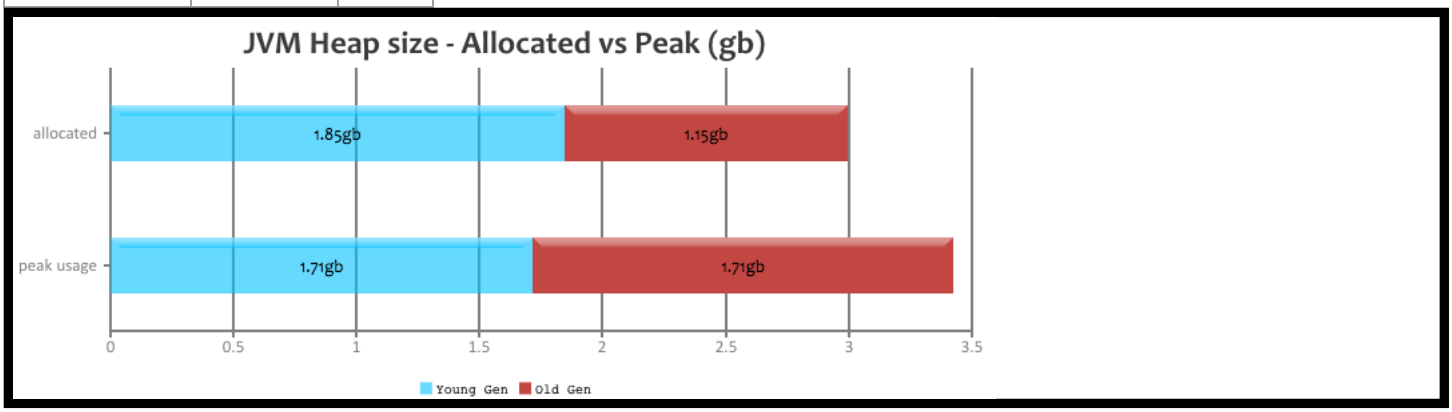
In **1 GC event(s)**, 'sys' time is greater than 'usr' time. It's not a healthy sign.

Timestamp	User Time (secs)	Sys Time (secs)	Real Time (secs)
2017-08-19T15:15:29	0.26	0.48	0.29

Read our recommendations to **reduce sys time (https://blog.gceasy.io/2016/12/11/sys-time-greater-than-user-time/)**

JVM Heap Size

Generation	Allocated	Peak
Young Generation	1.85 gb	1.71 gb
Old Generation	1.15 gb	1.71 gb
Total	3 gb	2.35 gb



Key Performance Indicators

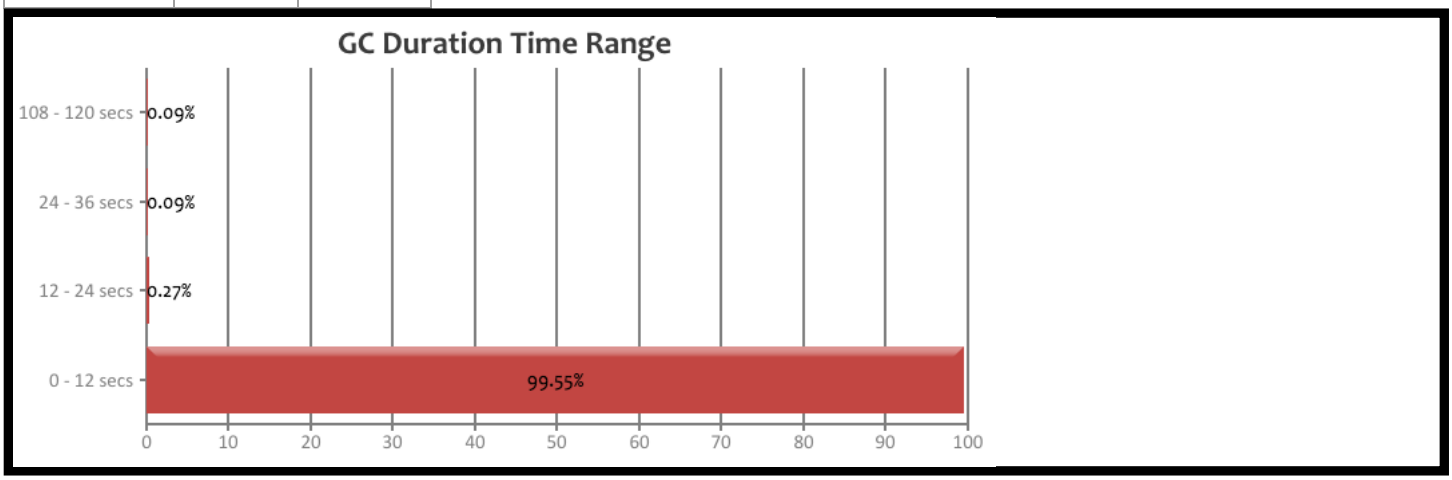
(Important section of the report. To learn more about KPIs, click here (https://blog.gceasy.io/2016/10/01/garbage-collection-kpi/))

1. **Throughput : 87.093%**
2. **Latency:**

Avg GC Time ❓	**482 ms**
Max GC Time ❓	**1 min 50 sec 320 ms**

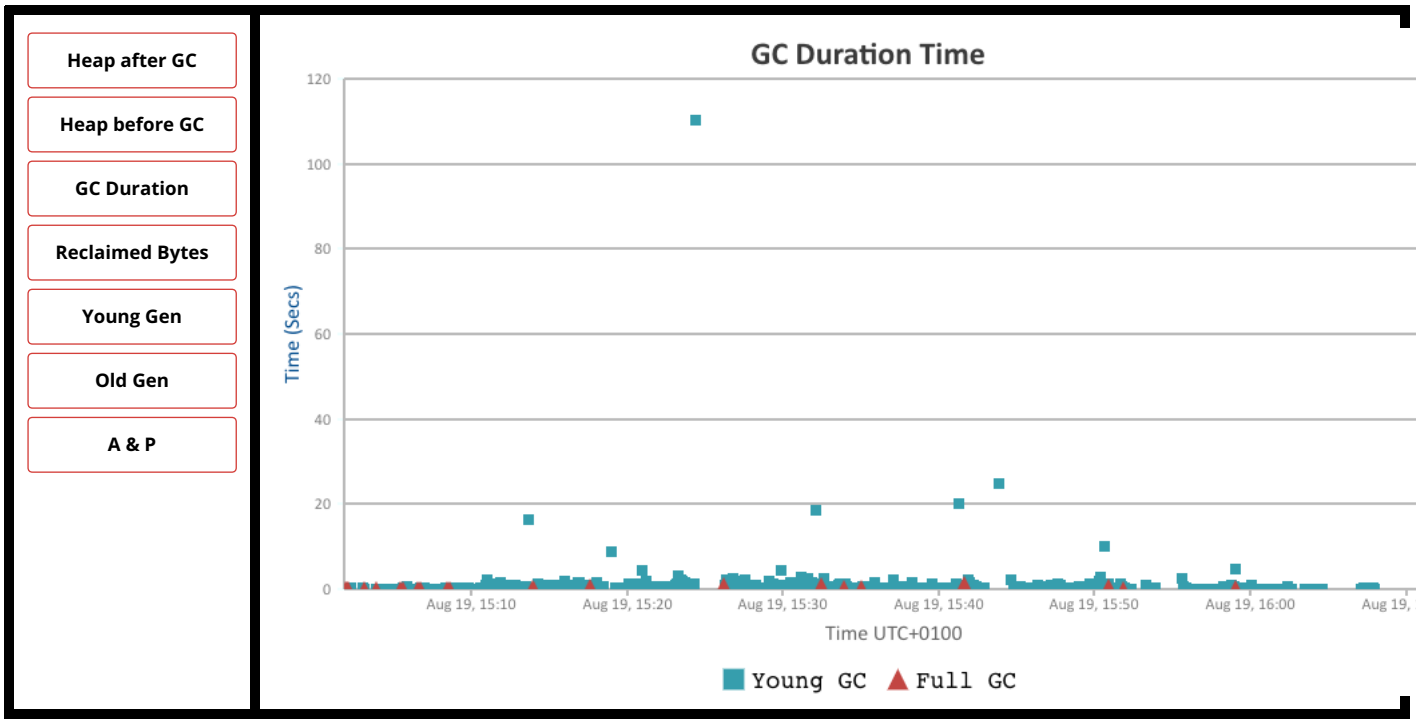
GC Duration Time Range ❓:

Duration (secs)	No. of GCs	Percentage
0 - 12	1102	99.548%
12 - 24	3	99.819%
24 - 36	1	99.91%
108 - 120	1	100.0%

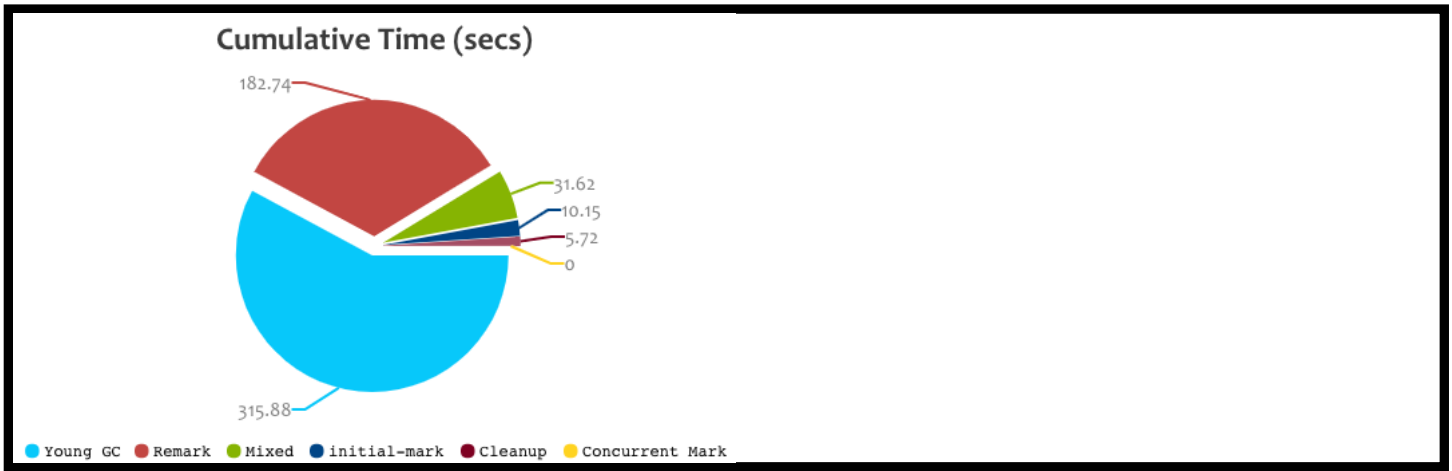


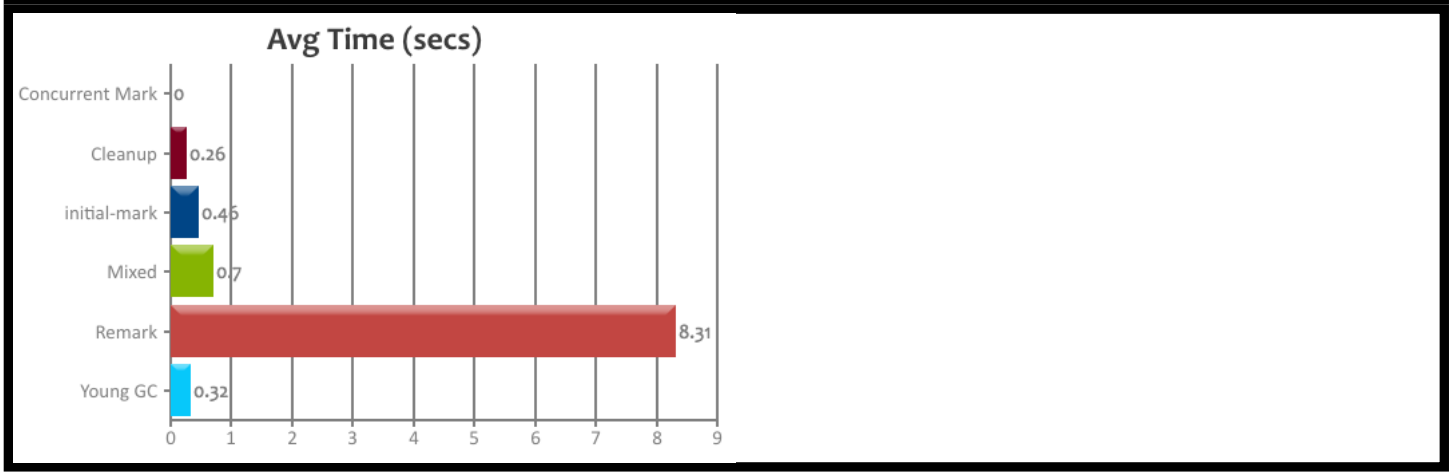
Interactive Graphs

(All graphs are zoomable)



G1 GC Statistics





	Remark	initial-mark	Mixed	Young GC	Concurrent Mark	Cleanup	Total
Count	22	22	45	999	22	22	1132
Total GC Time	3 min 2 sec 740 ms	10 sec 150 ms	31 sec 620 ms	5 min 15 sec 880 ms	0	5 sec 720 ms	9 min 6 sec 110 ms
Avg GC Time	8 sec 306 ms	461 ms	703 ms	316 ms	0	260 ms	482 ms
Avg Time std dev	23 sec 160 ms	690 ms	610 ms	925 ms	0	356 ms	3 sec 525 ms
Min/Max Time	0 / 1 min 50 sec 320 ms	0 / 2 sec 830 ms	0 / 2 sec 570 ms	0 / 24 sec 760 ms	0 / 0	0 / 1 sec 330 ms	0 / 1 min 50 sec 320 ms
Avg Interval Time	2 min 42 sec 957 ms	2 min 42 sec 772 ms	1 min 9 sec 966 ms	4 sec 239 ms	2 min 42 sec 957 ms	2 min 43 sec 179 ms	18 sec 648 ms

Object Stats

(These are good metrics to include in your performance reports)

Total created bytes	208.26 gb
Total promoted bytes	5.68 gb

Avg creation rate	50.4 mb/sec
Avg promotion rate	1.38 mb/sec

Memory Leak

No major memory leaks.

(**Note:** there are 8 flavours of OutOfMemoryErrors (https://tier1app.files.wordpress.com/2014/12/outofmemoryerror2.pdf). With GC Logs you can diagnose only 5 flavours of them(Java heap space, GC overhead limit exceeded, Requested array size exceeds VM limit, Permgen space, Metaspace). So in other words, your application could be still suffering from memory leaks, but need other tools to diagnose them, not just GC Logs.)

Consecutive Full GC

None.

Safe Point Duration

(To learn more about SafePoint duration, click here (https://blog.gceasy.io/2016/12/22/total-time-for-which-application-threads-were-stopped/))

Not Reported in the log.

GC Causes

(What events caused the GCs, how much time it consumed?)

Cause	Count	Avg Time	Max Time	Total Time	Time %
G1 Evacuation Pause	1023	311 ms	24 sec 243 ms	5 min 18 sec 534 ms	58.33%
Others	66	n/a	n/a	3 min 32 sec 937 ms	38.99%
GCLocker Initiated GC	28	259 ms	1 sec 93 ms	7 sec 265 ms	1.33%
G1 Humongous Allocation	7	899 ms	2 sec 800 ms	6 sec 292 ms	1.15%
Metadata GC Threshold	8	135 ms	488 ms	1 sec 82 ms	0.2%
Total	1132	n/a	n/a	9 min 6 sec 110 ms	100.0%

Tenuring Summary

Not reported in the log.

Command Line Flags

-XX:G1ReservePercent=20 -XX:GCLogFileSize=10485760 -XX:InitialHeapSize=3221225472 -XX:+ManagementServer -XX:MaxGCPauseMillis=200 -XX:MaxHeapSize=3221225472 -XX:NumberOfGCLogFiles=10 -XX:+PrintGC -XX:+PrintGCDateStamps -XX:+PrintGCDetails -XX:+PrintGCTimeStamps -XX:+UseCompressedClassPointers -XX:+UseCompressedOops -XX:+UseG1GC -XX:+UseGCLogFileRotation

Become a DevOps champion in your organization

(Best practises/tools)

- Use **fastThread.io** (https://fastthread.io/) to analyze thread dumps, core dumps and hs_err_pid files
- Do proactive Garbage Collection analysis on all your JVMs (not just one or two) using the GC log analysis API (https://blog.gceasy.io/2016/06/18/garbage-collection-log-analysis-api/)
- Always rotate the GC log files (https://blog.gceasy.io/2016/11/15/rotating-gc-log-files/) to prevent critical garbage collection data loss
- Use 'Pro' version (http://gceasy.io/pricing.jsp) for 10x fast, unlimited, secure usage

Do you like this report?